# WellnessBot

## Installation & User Guide

Version: v2.0

Application: WellnessBot — Rehabilitation Decision Support

Document Type: Appendix — Installation & User Guide

# Contents

# 1. System Overview

WellnessBot is a neuro-symbolic hybrid AI chatbot that provides patients with contextually appropriate, evidence-grounded exercise recommendations during arthroscopic knee rehabilitation. It adapts dynamically to each user's surgery type, recovery phase, reported symptoms, available equipment, and exercise history.

## Pipeline Architecture

```
User Input
→ [1] NLU Extraction        (OpenAI GPT-4o-mini)
→ [2] Dialog Merge          (Slot filling into ConversationState)
→ [3] CLARIFY Check         (Missing slots trigger question loop)
→ [4] User State Inference  (Derives phase_id, risk_flags)
→ [5] Rule Engine           (RECOMMEND / CLARIFY / FORBID / ESCALATE)
→ [6] Intervention Planner  (Exercise selection — RECOMMEND path only)
→ [7] RAG Retrieval         (BM25 evidence chunk lookup)
→ [8] LLM Response Generation (Patient-facing validated output)
→ User Output (Streamlit chat UI)
```

# 2. Prerequisites

Before installing WellnessBot, ensure the following are available on your machine:

| Requirement | Minimum Version | Notes |
|---|---|---|
| Python | 3.10+ | Check with python --version |
| pip | Latest | Bundled with Python |
| OpenAI API Key | — | OpenAI API Key is required for full system functionality |
| Internet connection | — | Internet connection is required for OpenAI API calls |

**Note:** WellnessBot has been tested on Windows 11. It is compatible with macOS and Linux, with minor command differences noted where applicable.

# 3. Installation

## 3.1 Extract the Zip Archive

The submission is delivered as a zip file. Extract it to a location of your choice.

After extraction, the folder layout will look like this:

**IRS-PM-2026-01-24-GRP7-WellnessBot/**

├── ProjectReport/

│ └── Installation_and_User_Guide ← this document

└── **SystemCode/** ← all runnable code lives here

├── .env.example

├── requirements.txt

├── run_streamlit.ps1

├── run_tests.ps1

├── data/

├── logs/

└── wellnessbot/

**Important:** All terminal commands in this guide must be run from inside the SystemCode/ folder unless otherwise stated.

Open a terminal (PowerShell on Windows, or Terminal on macOS/Linux) and navigate there:

**Windows (PowerShell):**

```
cd path\to\IRS-PM-2026-01-24-GRP7-WellnessBot\SystemCode
```

**macOS / Linux:**

```
cd path/to/IRS-PM-2026-01-24-GRP7-WellnessBot/SystemCode
```

## 3.2 Create a Virtual Environment

It is strongly recommended to use a virtual environment to isolate dependencies. Run the following from inside SystemCode/:

**Windows:**

```
python -m venv .venv

.venv\Scripts\activate
```

**macOS / Linux:**

```
python -m venv .venv

source .venv/bin/activate
```

You should see (.venv) prepended to your terminal prompt once the environment is active.

## 3.3 Install Dependencies

With the virtual environment active, install all required packages:

```
pip install -r requirements.txt
```

This installs the following packages:

| Package | Version | Purpose |
|---|---|---|
| streamlit | ≥ 1.36.0 | Web UI framework |
| openai | ≥ 1.40.0 | LLM API client (GPT-4o-mini) |
| pydantic | ≥ 2.7.0 | Data validation and schema enforcement |
| python-dotenv | ≥ 1.0.1 | Environment variable loading from .env |
| pyyaml | ≥ 6.0.1 | YAML configuration parsing |
| pytest | ≥ 8.2.0 | Test suite runner |
| ruff | ≥ 0.6.0 | Code linter |

## 3.4 Configure Environment Variables

The .env file must be created inside SystemCode/ (alongside requirements.txt). Create it by copying the provided example:

**Windows (PowerShell):**

```
Copy-Item .env.example .env
```

**macOS / Linux:**

```
cp .env.example .env
```

Open SystemCode/.env in a text editor and set the required values:

```
APP_VERSION=v2.0

LOGGING_ENABLED=1

LOG_DIR=./logs

OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

OPENAI_MODEL=gpt-4o-mini
```

**Security:** Please do not share your OPENAI Key

# 4. Running the Application

**Note:** All commands below assume your terminal is inside SystemCode/ with the virtual environment active.

## 4.1 Windows (PowerShell)

A convenience script is provided that automatically sets PYTHONPATH and launches Streamlit. Run it from inside SystemCode/:

```
.\run_streamlit.ps1
```

## 4.2 macOS / Linux

From inside SystemCode/, with the virtual environment active:

```
PYTHONPATH=.. streamlit run wellnessbot/Wellnessbot.py
```

## Accessing the Application

Once started, open your web browser and navigate to:

```
http://localhost:8501
```

Streamlit will also print the URL to the terminal. The application loads immediately — no build step is required.

# 5. User Guide

## 5.1 Launching the App

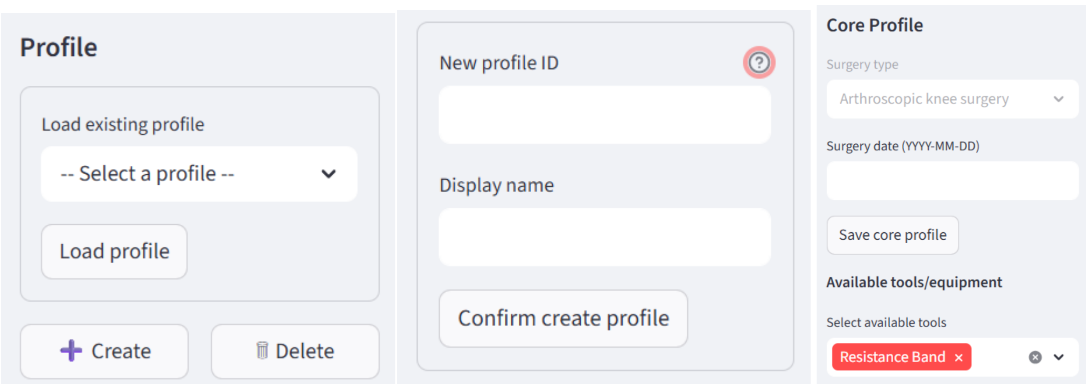
After running the start command (see section 4), open http://localhost:8501 in your browser. You will see the WellnessBot chat interface with a sidebar on the left.

## 5.2 Creating a User Profile

Before starting a conversation, you must select or create a user profile. The profile stores your surgery details and equipment list, so they persist across sessions.

**Steps:**

1. In the left sidebar, locate the Profile section.
2. If no profiles exist, click Create.
3. Enter the Display Name, then click “Confirm create profile” to initialise the profile.
4. After the profile is created, input the Surgery Date and Equipment Available in the profile settings.
5. To switch profiles, select the desired profile from the dropdown menu in the sidebar.



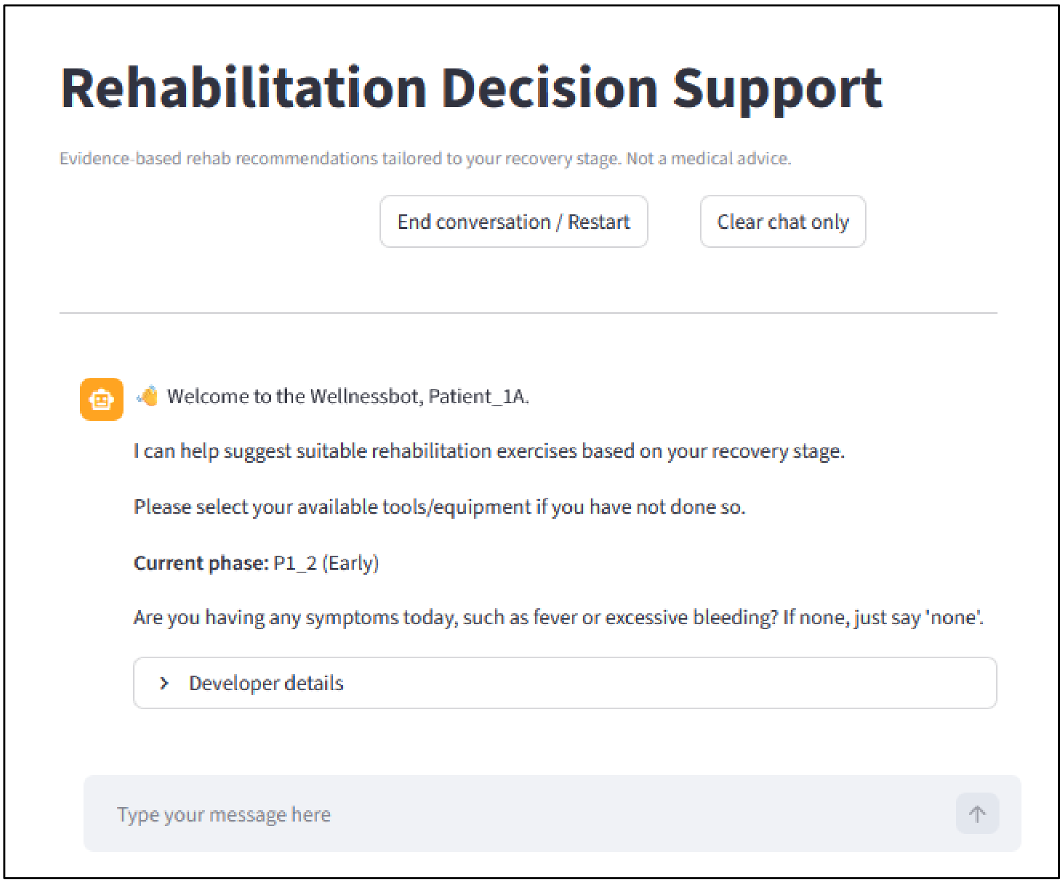

**Profile data** is stored locally in SystemCode/data/profiles/<profile_id>.json. It is not transmitted anywhere other than the OpenAI API for NLU processing.

## 5.3 Starting a Conversation

With a profile selected, type your message in the chat input box at the bottom of the screen and press Enter or click Send.

**Example opening messages:**

- "None"
- "I am having slight fever and no bleeding"




The bot will respond based on the information it has collected so far. If information is missing, it will ask follow-up questions.

## 5.4 Slot-Filling Dialogue Flow

WellnessBot collects the required user information before generating any exercise recommendation. If any required field is missing, the system issues a targeted follow-up question to ensure completeness of the input state.

For example, if the surgery date has not been provided during profile setup, WellnessBot will prompt the user to enter the surgery date before proceeding to the symptom assessment stage.

| Slot | What the Bot Asks | Example Answer |
|---|---|---|
| Surgery date / weeks | "When was your surgery or injury? Please tell me the date (YYYY-MM-DD) " | "2025-12-01" |
| Symptoms | "Are you having any symptoms today, such as fever or excessive bleeding? If none, just say “none”." | "I am having fever" / "None" |
| Pain score | "How would you rate the pain from 0 to 3?" | Numeric input (e.g., 1) or selection via UI buttons |
| Swelling score | "How would you rate the swelling from 0 to 3? " | Numeric input (e.g., 0) or selection via UI buttons |

**Tips:**

- If you have already set your surgery type and date in your profile, the bot may skip those questions.
- The bot remembers your answers within a session and will not re-ask completed slots.

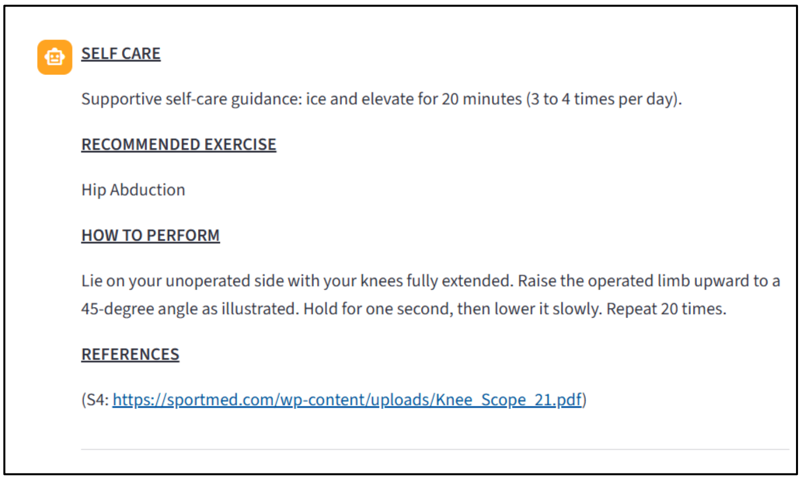
## 5.5 System Responses

Once sufficient information is collected, WellnessBot generates one of four responses:

### RECOMMEND — Exercise Recommendation

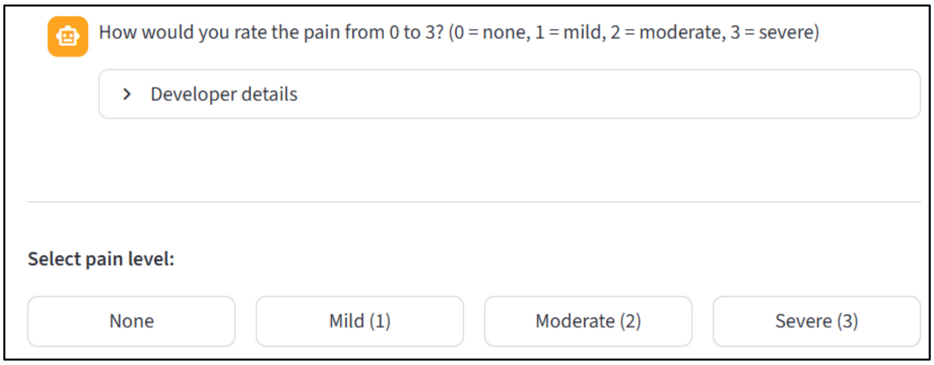
The most common response. The bot provides:

- Self Care (if applicable) — ice/elevation advice based on your symptoms
- Recommended Exercise — exercise name, position, equipment needed, and caution notes
- How To Perform — one grounded instruction line from the clinical evidence database
- References — source citations with links



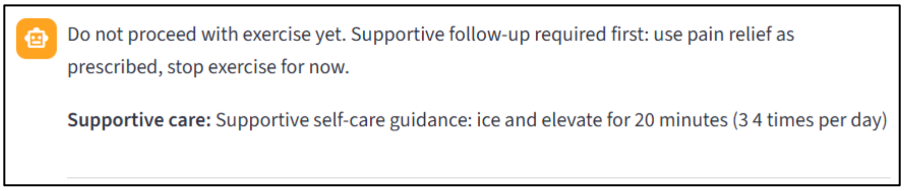

### CLARIFY — Follow-up Question (Internal Step)

The bot asks for one missing piece of information before it can make a recommendation. A list of possible exercises may also be shown.



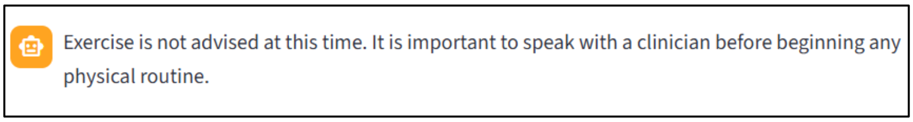
### FORBID — Exercise Blocked

The bot explains why an exercise cannot be recommended at this time (e.g., your swelling score is too high). No exercise is provided.




### ESCALATE — Seek Clinical Help

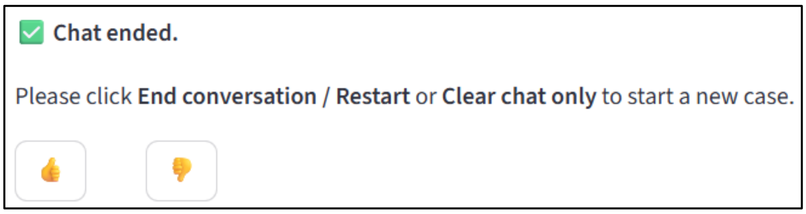
If red-flag symptoms are detected (e.g., fever, active bleeding, extreme swelling), the bot advises you to contact your healthcare provider immediately. No exercise is recommended.




## 5.6 Providing Feedback

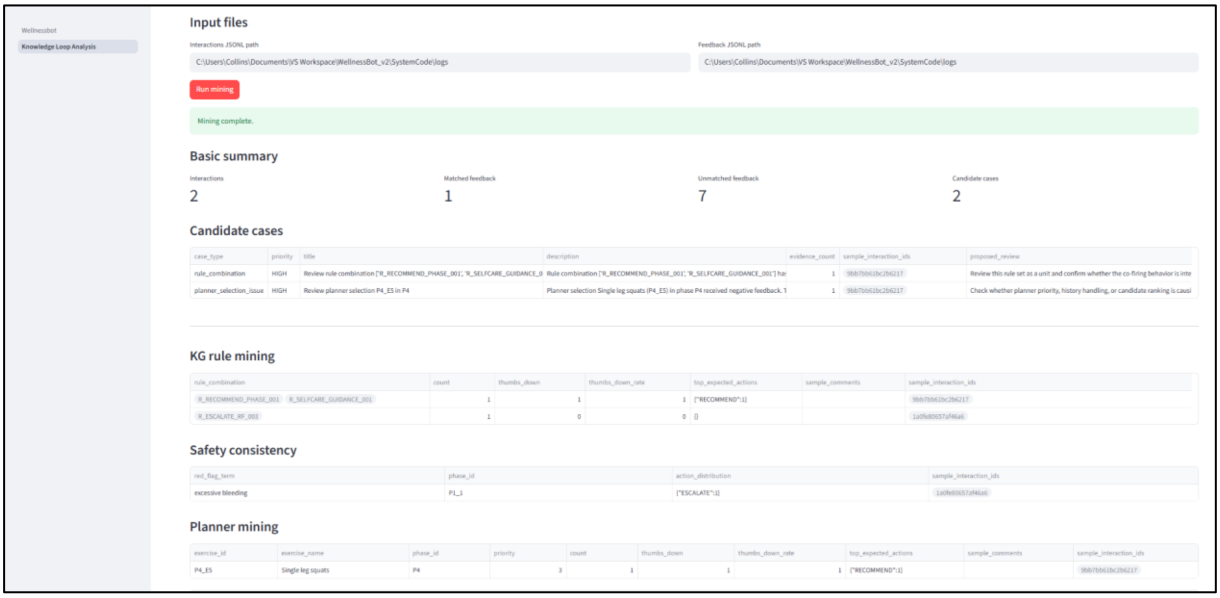
After each final system response (RECOMMEND, FORBID, or ESCALATE), a feedback prompt is presented to the user in the form of a thumbs-up or thumbs-down selection. Users may indicate whether the response was helpful or not through this interface.

All feedback is recorded as part of the system's audit trail, together with the corresponding decision, input state, and reasoning outputs. This information is subsequently used for developer analysis to assess system performance, identify failure patterns, and support targeted refinement of the knowledge graph, rule set, and planner logic through the offline knowledge loop.



## 5.7 Knowledge Loop Analysis

The Knowledge Loop Analysis page is used to evaluate system behaviour using interaction logs and user feedback.




**Steps:**

1. Select Input Files

   Ensure the interaction logs and feedback logs are correctly loaded.

2. Run Mining

   Click “Run Mining” to process logs and generate results.

3. Review Basic Summary

   View key metrics: total interactions, feedback coverage, and refinement candidates.

4. Analyse Refinement Candidates

   Review flagged cases with issue summaries, interaction IDs, and suggested actions.

   These highlight potential problems such as rule conflicts, negative feedback patterns, or suboptimal planner decisions.

5. Deep Dive into Mining Views

   - KG Rule Mining: Identify problematic rule combinations
   - Safety Consistency: Check if red-flag conditions trigger correct actions
   - Planner Mining: Evaluate exercise selection behaviour

This workflow supports systematic identification of system weaknesses and guides manual, developer-driven refinement of the knowledge graph, rules, and planner, without affecting runtime logic.

# 6. Environment Variables Reference

| Variable | Required | Default | Description |
| --- | --- | --- | --- |
| OPENAI_API_KEY | Yes | — | OpenAI API key for NLU and LLM response generation |
| OPENAI_MODEL | No | gpt-4o-mini | OpenAI model for NLU extraction |
| LOGGING_ENABLED | No | 1 | Set to 0 to disable interaction logging |
| LOG_DIR | No | ./logs | Directory for daily JSONL interaction log files |
| APP_VERSION | No | v2 | Version tag written into audit log entries |

# 7. Troubleshooting

## 7.1 Script not found / CommandNotFoundException

Ensure you are running the command from inside SystemCode/, not from the repo root or elsewhere:

```
cd path\to\IRS-PM-2026-01-24-GRP7-WellnessBot\SystemCode .\run_streamlit.ps1
```

## 7.2 PowerShell script execution blocked

Run this once to allow local scripts, then retry:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

## 7.3 Application does not start

- Ensure your virtual environment is active ((.venv) in your terminal prompt).
- Ensure all dependencies are installed: pip install -r requirements.txt.
- Ensure you are running the command from inside SystemCode/.

## 7.4 ModuleNotFoundError: No module named 'SystemCode'

- PYTHONPATH must point to the parent of SystemCode/ (i.e., IRS-PM-2026-01-24-GRP7-WellnessBot/).
- On macOS/Linux, use PYTHONPATH=.. when running commands manually.
- On Windows, run_streamlit.ps1 sets this automatically — use the script rather than running streamlit directly.

## 7.5 openai.AuthenticationError or 401 Unauthorized

- Your OPENAI_API_KEY in SystemCode/.env is missing or incorrect.
- Verify the key at platform.openai.com.

## 7.6 Profile not saving

- Ensure the SystemCode/data/profiles/ directory exists. It is created automatically on first run.
- Check write permissions on the SystemCode/data/ directory.

## 7.7 Logs not being written

- Ensure LOGGING_ENABLED=1 in SystemCode/.env.
- Ensure the SystemCode/logs/ directory exists and is writable.